ASSIGNMENT 2.1

Concurrent Task Daemonization using Threads and Queues

Student	[Student Name]	Roll No	[Roll No]
Course	21CSE525T - Agent Technology	Project	Placement Preparation Agent

Unit 2 - Background daemons with a shared queue and a lock. File: assignments/a2_1/concurrent_agent.py

1. Objective

Run several polling jobs in the background so that the agent's main loop stays responsive, and keep shared state safe when several threads update it.

2. Architecture

Three **producer daemons** poll the environment: CodingMonitor (coding platform), DriveMonitor (placement calendar) and AptitudeMonitor (aptitude engine). Each pushes percepts onto a queue.Queue.

One **consumer thread**, StateUpdater, takes percepts off the queue and applies them to shared_state inside a threading.Lock critical section.

The **main thread** prints a snapshot of the shared state every 0.7 s for 8 cycles. Its printing never waits for the workers.

A threading.Event (shutdown) stops all daemons cleanly. Each daemon waits on the event instead of sleeping, so it wakes up immediately on shutdown.

3. Concurrency safety

- **Queue as the only channel:** producers never touch shared_state; they only put items on the queue, which is thread-safe.
- **Single writer behind a lock:** shared_state is changed only in StateUpdater, inside with state_lock.
- **Read under lock:** the main loop copies the state under the same lock, so it never sees a half-updated record.
- **Clean shutdown:** shutdown.set() and join(timeout) instead of killing threads.

4. Implementation

```
def coding_platform_monitor(interval=0.8):
    rng = random.Random(11)
    while not shutdown.is_set():
        t0 = time.perf_counter()
        solved = rng.randint(0, 3)
        score = rng.randint(40, 95)
        percept_queue.put({"src": "CODING", "solved": solved, "score": score})
        log("CodingMonitor", f"polled platform -&gt; +{solved} solved, score={score}%")
        with latency_lock:
            latency_log.append(("CodingMonitor", (time.perf_counter() - t0) * 1000))
        shutdown.wait(interval)

def state_updater():
    while not shutdown.is_set() or not percept_queue.empty():
        try:
            p = percept_queue.get(timeout=0.3)
        except queue.Empty:
            continue
        with state_lock:                        # &lt;-- critical section
            if p["src"] == "CODING":
                shared_state["problems_solved"] += p["solved"]
                shared_state["coding_score"] = p["score"]
            elif p["src"] == "APTITUDE":
                shared_state["aptitude_score"] = p["score"]
            elif p["src"] == "DRIVE":
                shared_state["open_drives"] += 1
            shared_state["updates_applied"] += 1
```

```
        log("StateUpdater", f"applied {p['src']} percept  (queue={percept_queue.qsize()})")
        percept_queue.task_done()
```

5. Result

In the recorded run the main loop completed all 8 cycles while the daemons ran. Final state: problems_solved = 15, updates_applied = 16 (each queued percept applied exactly once). Latency averaged about 0.12 to 0.14 ms per poll for each worker.

Execution output (log file)

Captured from **logs/2.1_concurrent_agent.log** (python3 concurrent_agent.py). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 2.1_concurrent_agent
# command: python3 concurrent_agent.py
# exit code: 0
# lines: 74

================================================================================
 PLACEMENT PREP AGENT :: CONCURRENT DAEMON INFRASTRUCTURE (Assignment 2.1)
================================================================================

[10:40:15.148] [CodingMonitor     ] polled platform -&gt; +3 solved, score=95%
[10:40:15.148] [MAIN              ] started daemon thread 'CodingMonitor' (daemon=True)
[10:40:15.148] [DriveMonitor      ] no new drives this cycle
[10:40:15.148] [MAIN              ] started daemon thread 'DriveMonitor' (daemon=True)
[10:40:15.148] [AptitudeMonitor   ] aptitude engine sync -&gt; score=46%
[10:40:15.148] [MAIN              ] started daemon thread 'AptitudeMonitor' (daemon=True)
[10:40:15.148] [StateUpdater      ] applied CODING percept  (queue=1)
[10:40:15.148] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:15.148] [MAIN              ] started daemon thread 'StateUpdater' (daemon=True)

[10:40:15.148] [MAIN-EVENTLOOP    ] cycle 1: UI responsive | code=95% apt=46% solved=3 drives=0
[10:40:15.853] [MAIN-EVENTLOOP    ] cycle 2: UI responsive | code=95% apt=46% solved=3 drives=0
[10:40:15.953] [CodingMonitor     ] polled platform -&gt; +3 solved, score=68%
[10:40:15.953] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:16.153] [AptitudeMonitor   ] aptitude engine sync -&gt; score=39%
[10:40:16.153] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:16.453] [DriveMonitor      ] no new drives this cycle
[10:40:16.558] [MAIN-EVENTLOOP    ] cycle 3: UI responsive | code=68% apt=39% solved=6 drives=0
[10:40:16.758] [CodingMonitor     ] polled platform -&gt; +1 solved, score=51%
[10:40:16.758] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:17.158] [AptitudeMonitor   ] aptitude engine sync -&gt; score=82%
[10:40:17.158] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:17.263] [MAIN-EVENTLOOP    ] cycle 4: UI responsive | code=51% apt=82% solved=7 drives=0
[10:40:17.562] [CodingMonitor     ] polled platform -&gt; +3 solved, score=80%
[10:40:17.562] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:17.758] [DriveMonitor      ] no new drives this cycle
[10:40:17.968] [MAIN-EVENTLOOP    ] cycle 5: UI responsive | code=80% apt=82% solved=10 drives=0
[10:40:18.163] [AptitudeMonitor   ] aptitude engine sync -&gt; score=91%
[10:40:18.164] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:18.366] [CodingMonitor     ] polled platform -&gt; +1 solved, score=46%
[10:40:18.366] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:18.673] [MAIN-EVENTLOOP    ] cycle 6: UI responsive | code=46% apt=91% solved=11 drives=0
[10:40:19.060] [DriveMonitor      ] new placement drive detected -&gt; Wipro
[10:40:19.060] [StateUpdater      ] applied DRIVE percept  (queue=0)
[10:40:19.164] [AptitudeMonitor   ] aptitude engine sync -&gt; score=48%
[10:40:19.165] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:19.169] [CodingMonitor     ] polled platform -&gt; +3 solved, score=59%
[10:40:19.169] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:19.374] [MAIN-EVENTLOOP    ] cycle 7: UI responsive | code=59% apt=48% solved=14 drives=1
[10:40:19.974] [CodingMonitor     ] polled platform -&gt; +1 solved, score=45%
[10:40:19.974] [StateUpdater      ] applied CODING percept  (queue=0)
[10:40:20.079] [MAIN-EVENTLOOP    ] cycle 8: UI responsive | code=45% apt=48% solved=15 drives=1
[10:40:20.166] [AptitudeMonitor   ] aptitude engine sync -&gt; score=93%
[10:40:20.166] [StateUpdater      ] applied APTITUDE percept  (queue=0)
[10:40:20.366] [DriveMonitor      ] new placement drive detected -&gt; Wipro
[10:40:20.366] [StateUpdater      ] applied DRIVE percept  (queue=0)
[10:40:20.779] [CodingMonitor     ] polled platform -&gt; +0 solved, score=78%
[10:40:20.779] [StateUpdater      ] applied CODING percept  (queue=0)

--------------------------------------------------------------------------------
[10:40:20.784] [MAIN              ] shutdown signal issued to all daemons ...
[10:40:20.785] [MAIN              ] thread 'CodingMonitor' joined cleanly (alive=False)
[10:40:20.785] [MAIN              ] thread 'DriveMonitor' joined cleanly (alive=False)
[10:40:20.785] [MAIN              ] thread 'AptitudeMonitor' joined cleanly (alive=False)
```

```
[10:40:21.083] [MAIN             ] thread 'StateUpdater' joined cleanly (alive=False)

================================================================================
 FINAL SHARED STATE (no race conditions):
   coding_score       = 78
   aptitude_score     = 93
   open_drives        = 2
   problems_solved    = 15
   updates_applied    = 16

 EXECUTION LATENCY REPORT (ms):
   CodingMonitor      n=8   avg=0.1260 max=0.1772
   DriveMonitor       n=5   avg=0.1253 max=0.2045
   AptitudeMonitor    n=6   avg=0.1224 max=0.1750
   MAIN-EVENTLOOP     n=8   avg=0.1371 max=0.1880
================================================================================
```

6. Viva points

Why a queue instead of a shared list? A queue handles its own locking and blocking, so producers and consumer cannot corrupt it.

Why one writer? Two threads writing the same dictionary can lose updates; a single writer under a lock keeps the count exact (16 updates applied).

Daemon threads stop when the main program exits; the Event makes that stop orderly.

7. Conclusion

Daemons plus a locked queue let the agent keep sensing while it reasons. The updates_applied count matching the number of queued percepts is the evidence that no update was lost.